

ENTREGABLE N.º 2

Diseño de Sistemas de Información

Residencia360 — Sistema de Información para la Unidad Residencial «Altos de Santa Clara»

Nombre del proyecto	Residencia360
Integrantes	Juan Camilo Cardona Sánchez · Yesid Ardila Elejalde
Docente	[Nombre del docente]
Curso	Diseño de Sistemas de Información
Fecha	Mayo de 2026
Institución	Instituto Tecnológico Metropolitano (ITM)

Contenido

1. Propósito y relación con el Entregable N.º 1

El presente documento corresponde al Entregable N.º 2 del proyecto Residencia360 y constituye una continuación estructurada del Entregable N.º 1, no una entrega independiente. Su objetivo es evidenciar el avance técnico del sistema y, simultáneamente, consolidar el trabajo previo: los requisitos, las reglas de negocio, el story mapping, las historias de usuario y el diagrama de clases definidos en la primera entrega se mantienen como base y se refinan hasta convertirse en una propuesta técnica concreta —arquitectura C4, patrones de diseño, prototipo funcional, diagramas de actividad y análisis de principios SOLID.

La relación entre ambos momentos del proyecto se materializa de forma explícita a lo largo del documento. El Entregable N.º 1 aportó la base de análisis; el Entregable N.º 2 toma esos insumos, corrige sus debilidades —identificadas en una auditoría previa— y los proyecta sobre decisiones de diseño justificadas. En particular, el diagrama de clases inicial se refina y se conecta con el nivel 4 del modelo C4; el story mapping y el MVP sustentan las historias de usuario implementadas; y el prototipo Figma de la primera entrega sirve de base al prototipo funcional de esta.

Trazabilidad de la consolidación. Cada ajuste presentado en la sección 2 responde a un hallazgo concreto de la auditoría del Entregable N.º 1. A lo largo del texto se referencian dichos hallazgos con la nomenclatura **H-xx**, de modo que sea verificable que ningún elemento previo se abandona: todo se integra, se refina y se materializa en esta fase.

1.1 Glosario y unificación terminológica

Para eliminar la ambigüedad detectada en la primera entrega (donde un mismo concepto recibía varios nombres), se adopta la siguiente terminología única en todos los artefactos:

Término oficial	Definición	Sustituye a
Vigilante	Usuario con rol de control de acceso que opera la portería.	Guarda, Portería
PreRegistroVisita	Solicitud de ingreso de un visitante autorizada por un residente.	Visita, pre-registro
Visitante	Persona externa que ingresa a la unidad asociada a un pre-registro.	—
PQRS	Petición, queja, reclamo o sugerencia con seguimiento y SLA.	ticket
QRToken	Credencial temporal de un solo uso para validar el ingreso.	QR, escarpela
Residente	Usuario propietario o arrendatario asociado a un apartamento.	—

2. Refinamiento y consolidación del Entregable N.º 1

Esta sección evidencia los ajustes realizados sobre los artefactos de la primera entrega y cómo se consolidan dentro del Entregable N.º 2. La Tabla 1 resume cada ajuste con su estado anterior, su forma refinada, el motivo y el hallazgo de auditoría que lo origina.

2.1 Tabla de ajustes realizados frente a la entrega anterior

Tabla 1. Ajustes realizados frente al Entregable N.º 1.

Elemento ajustado	Cómo estaba antes	Cómo quedó refinado	Motivo / Hallazgo
Diagrama de clases (herencia)	Persona y Usuario coexistían sin jerarquía clara; los roles colgaban de forma ambigua.	Persona 1—1 Usuario; Residente, Vigilante y Administrador especializan a Usuario por generalización.	Modelado correcto (H-14)
Multiplicidades y composición	Asociaciones sin multiplicidad; no se distinguía composición de asociación.	Multiplicidad en todos los extremos; composición (rombo) en Unidad → Torre → Apartamento.	Rigor UML (H-15)
Alcance del modelo	El diagrama incluía módulos sin requisito (paquetería).	El dominio se acota al MVP; paquetería se difiere como extensión futura.	Coherencia de alcance (H-16)
Estados como enumeraciones	Los estados de visita y PQRS eran texto suelto en las reglas.	Se tipifican como enumeraciones (EstadoVisita, EstadoPQRS, Prioridad...) en el dominio.	Vínculo regla-modelo (H-05)
Arquitectura	No existía ninguna vista arquitectónica.	Se incorpora el modelo C4 completo (niveles 1 a 4) derivado del E1.	Arquitectura (H-19)
Story mapping	Sección con placeholder; sin imagen ni enlace.	Story map reconstruido (backbone, walking skeleton, releases) con MVP unificado.	Artefacto ausente (H-08, H-10)
Fuera de alcance	Apartado declarado y vacío.	Se define explícitamente lo excluido del MVP y su justificación.	Delimitación (H-09)
Historias de usuario	8 de 15 historias sin criterios; remitían a herramienta sin enlace.	Todas las historias del MVP con Gherkin (flujo principal); alternativos en el E3.	Complejidad (H-11)
Trazabilidad	Matriz parcial: faltaban RU, RN, RNF y componentes.	Matriz extendida y bidireccional (RU ↔ RF ↔ RN ↔ HU ↔ Pantalla ↔ Componente C4).	Trazabilidad (H-01, H-21)
RNF verificables	Metas sin método de verificación.	Cada RNF con métrica, método y umbral; vinculados a decisiones C4.	Verificabilidad (H-02)
Patrones y SOLID	Inexistentes en el E1.	5 patrones GoF justificados y análisis SOLID sobre componentes reales.	Avance técnico (H-27)

2.2 Story mapping y MVP consolidados

El story mapping se reconstruye con un backbone de actividades de usuario (de izquierda a derecha, en orden de uso) y, bajo cada actividad, las historias ordenadas por prioridad (de arriba hacia abajo). La primera franja horizontal constituye el walking skeleton —el MVP— y las siguientes, los incrementos posteriores.

Backbone (actividades). A1 Accesos y seguridad · A2 Comunicación · A3 PQRS y mantenimiento · A4 Zonas comunes · A5 Cartera y pagos · A6 Reportes.

MVP unificado (primera entrega). Se resuelve la desalineación detectada (H-10) fijando un MVP nítido y verificable, idéntico en el mapa, en la lista de historias y en la matriz de trazabilidad:

- Autenticación y roles (RF-01) — habilita todo lo demás.
- Visitas: pre-registro, validación en portería y registro de salida (RF-04, RF-06, RF-07).
- Anuncios y comunicados (RF-11).
- PQRS con estados y SLA (RF-13, RF-14).
- Reservas de zonas comunes con calendario y reglas (RF-15, RF-16).
- Reportes básicos (RF-20).

Incrementos posteriores (Release 2+). Parqueaderos de visitantes (RF-10), cartera y validación de pagos manuales (RF-17, RF-18) y configuración avanzada de reglas y SLA.

2.2.1 Fuera de alcance del MVP (delimitación explícita)

Para cerrar el vacío de la primera entrega (H-09), se excluye explícitamente del MVP, con su justificación:

Excluido del MVP	Justificación
Pasarela de pagos en línea real	El MVP valida pagos de forma manual; la integración transaccional exige cumplimiento PCI y un proveedor, fuera del tiempo del curso.
Biometría / reconocimiento facial en portería	El control por QRToken cubre la necesidad de trazabilidad sin el costo y los riesgos de privacidad de la biometría.
Aplicación móvil nativa	La SPA responsive cubre móvil y escritorio; una app nativa es un esfuerzo independiente.
Integración contable / ERP	La cartera del MVP es de consulta y registro; la conciliación contable es un sistema aparte.

2.3 Matriz de trazabilidad extendida

La matriz de trazabilidad se reconstruye de forma completa y bidireccional, incorporando reglas de negocio y el componente del modelo C4 (nivel 3) responsable de cada capacidad. Esto corrige los hallazgos H-01 y H-21 y constituye la columna vertebral de coherencia entre los tres entregables.

Tabla 2. Matriz de trazabilidad extendida del MVP.

RU	RF	Reglas	Historia	Pantalla	Componente C4 (N3)
RU-01	RF-01, RF-02	RN-12	HU-01, HU-02	S-01, S-02	AuthController / AuthMiddleware

RU	RF	Reglas	Historia	Pantalla	Componente C4 (N3)
RU-02	RF-04, RF-05	RN-01, RN-02	HU-03, HU-04	S-03	AccesoController / AccesoService
RU-03	RF-06, RF-07	RN-03	HU-05, HU-06	S-04	AccesoController / AccesoService
RU-05	RF-11, RF-12	RN-11	HU-07, HU-08	S-05	ComunicadoController / ComunicadoService
RU-06	RF-13, RF-14	RN-09, RN-10	HU-09, HU-10	S-06, S-07	PqrsController / PqrsService
RU-07	RF-15, RF-16	RN-06, RN-07, RN-08	HU-11, HU-12	S-08, S-09	ReservaController / ReservaService
RU-10	RF-20	RN-12	HU-15	S-12	ReporteController / ReporteService
RU-11	RF-19	RN-11	HU-07, HU-09	(transversal)	NotificacionService (Observer)
RU-12	RF-21	RN-12	HU-01, HU-05, HU-10	Todas	AuditInterceptor / IAuditRepo

Nota: RU-04 (parqueaderos), RU-08 (cartera/pagos) y RU-09 (proveedores) quedan trazados en el alcance Release 2+ y se omiten de la matriz del MVP por delimitación explícita (§2.2.1).

3. Arquitectura del sistema (modelo C4)

La arquitectura se documenta con el modelo C4 (Context, Containers, Components, Code) propuesto por Simon Brown, que describe el sistema en cuatro niveles de zoom progresivo. Cada nivel se deriva de los insumos del Entregable N.º 1 —actores, requisitos, reglas e historias— de modo que la arquitectura no aparece desconectada del análisis previo, sino como su consecuencia natural. Este apartado corrige el hallazgo H-19 (ausencia de vista arquitectónica).

Estilo arquitectónico. Residencia360 adopta una arquitectura web cliente-servidor de tres contenedores (SPA, API y base de datos) y, dentro de la API, una organización en capas (presentación, aplicación, dominio y persistencia). Esta elección honra el RNF-07 (arquitectura por capas) de la primera entrega y habilita directamente los principios SOLID y los patrones que se justifican más adelante.

3.1 Nivel 1 — Diagrama de contexto

El nivel de contexto sitúa a Residencia360 como una caja única y muestra quién lo usa y con qué sistemas externos interactúa. Los actores corresponden exactamente a los definidos en el Entregable N.º 1 (Residente, Vigilante, Administrador y Consejo), y el único sistema externo del MVP es el proveedor de correo para las notificaciones (RF-19).

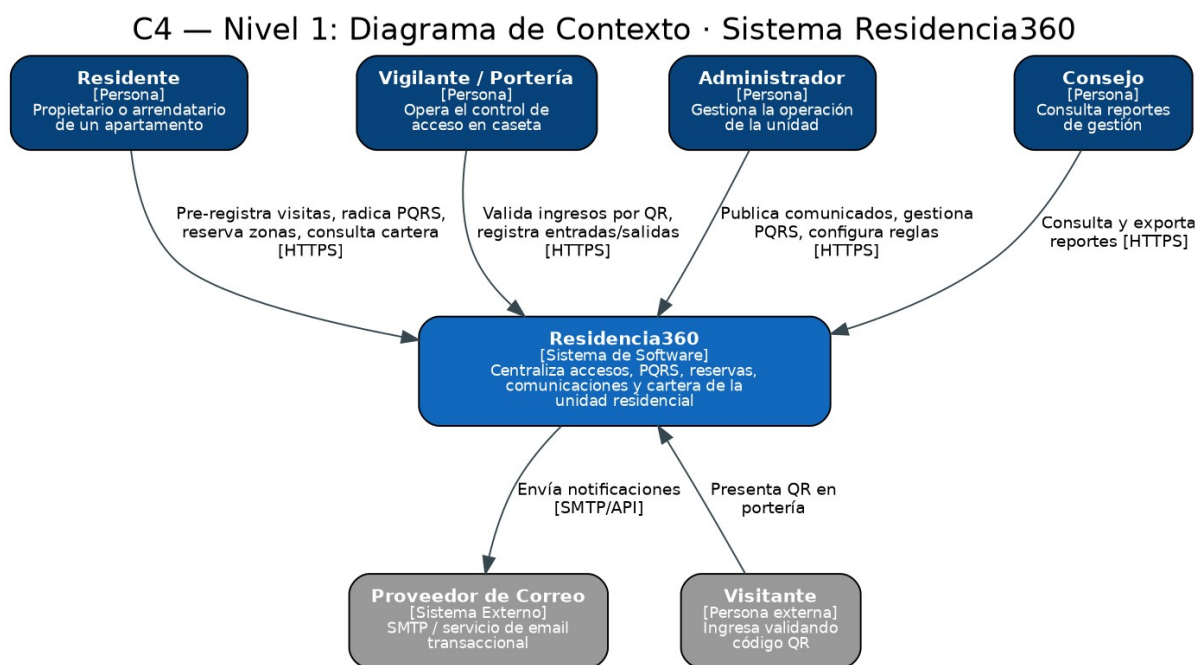


Figura 1. Diagrama de contexto del sistema Residencia360 (C4 nivel 1).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

Relación con el E1. Los cuatro actores provienen del modelo verbal; el visitante aparece como persona externa que valida su QRToken en portería (HU-05), y el proveedor de correo materializa el requisito de notificaciones (RU-11, RF-19).

3.2 Nivel 2 — Diagrama de contenedores

El nivel de contenedores descompone el sistema en unidades desplegadas por separado. Se definen tres contenedores: una aplicación web de página única (SPA) que provee la interfaz responsive para los cuatro roles; una API de aplicación que expone los servicios REST, orquesta los casos de uso y concentra la lógica de negocio y la seguridad; y una base de datos relacional que persiste todo el estado del sistema.

C4 — Nivel 2: Diagrama de Contenedores · Residencia360

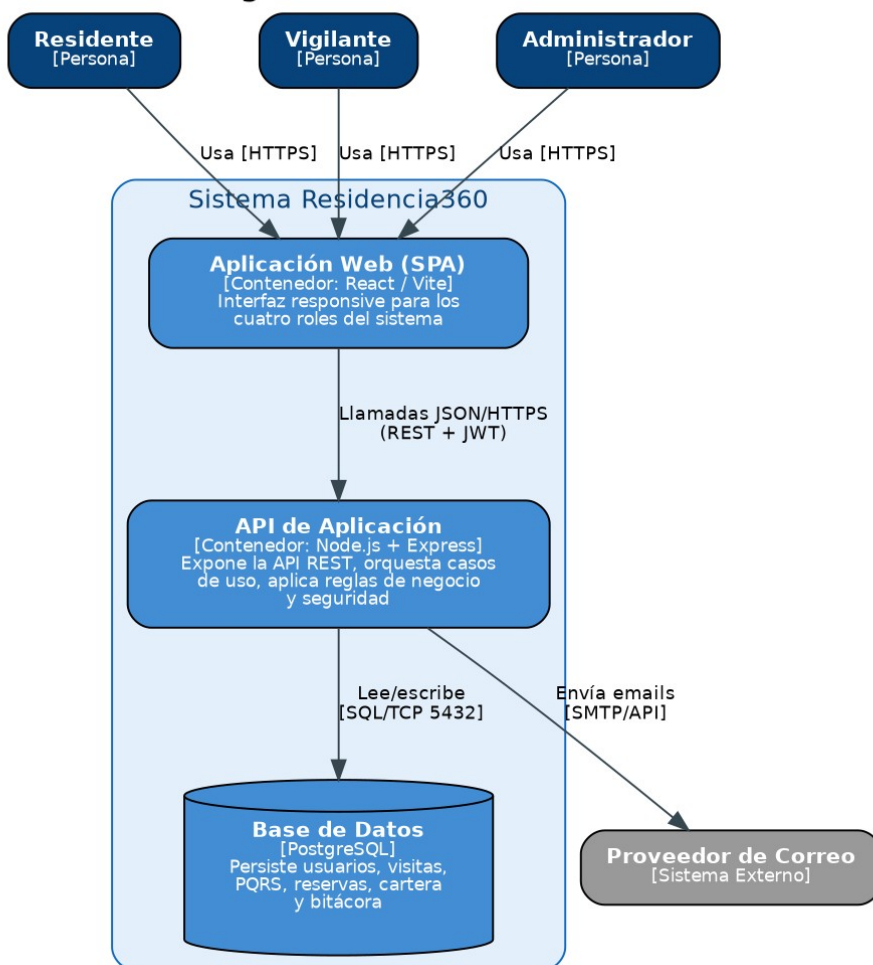


Figura 2. Diagrama de contenedores de Residencia360 (C4 nivel 2).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

Decisiones y RNF. La separación SPA/API permite cumplir el RNF-05 (interfaz responsive) sin acoplar la presentación a la lógica; la autenticación por token (JWT) en la frontera de la API soporta el RNF-01 (seguridad y sesión); y la base de datos relacional habilita la integridad referencial que exige la regla RN-01 (toda visita asociada a un apartamento y un residente).

3.3 Nivel 3 — Diagrama de componentes

El nivel de componentes abre el contenedor de la API y muestra su organización interna en capas. La capa transversal (middleware) resuelve autenticación, manejo de errores y auditoría;

los controladores traducen las peticiones HTTP; los servicios de aplicación orquestan los casos de uso; el modelo de dominio concentra las entidades, las reglas y las máquinas de estado; y los repositorios abstraen la persistencia. El servicio de notificaciones actúa como observador de los eventos de dominio.

C4 — Nivel 3: Diagrama de Componentes · Contenedor «API de Aplicación»

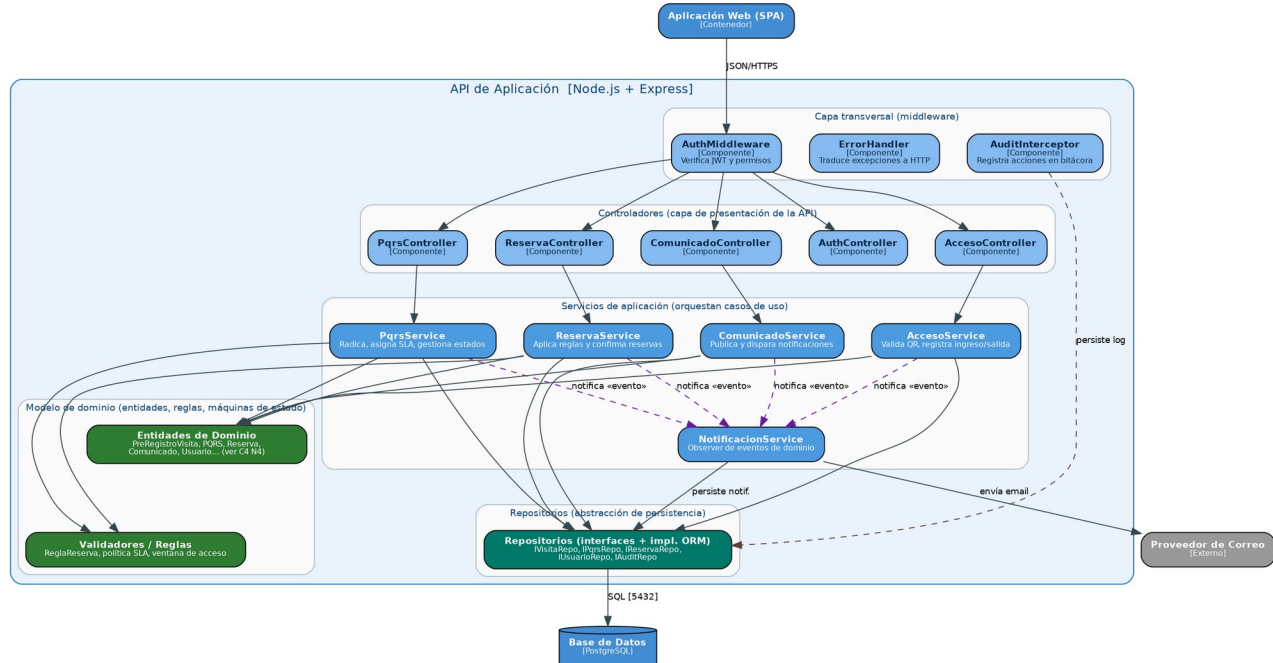


Figura 3. Diagrama de componentes del contenedor API (C4 nivel 3).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

Preparación para patrones y SOLID. Esta estructura por capas no es decorativa: cada frontera habilita un principio o un patrón. La interfaz `IRepositorio` entre servicios y persistencia habilita la inversión de dependencias (DIP) y el patrón que la implementa; el `NotificacionService` desacoplado habilita el patrón `Observer`; y la `PoliticaSLA` del dominio habilita el patrón `Strategy`. La sección 4 desarrolla cada uno.

3.4 Nivel 4 — Modelo de dominio (clases)

El nivel de código corresponde al diagrama de clases refinado del Entregable N.º 1. Se organiza por contextos delimitados (identidad y acceso, estructura residencial, accesos y visitantes, PQRS, reservas, comunicaciones y auditoría) y aplica las correcciones de la auditoría: separación `Persona/Usuario` con generalización correcta (H-14), multiplicidades y composición explícitas (H-15), acotación al alcance del MVP (H-16) y entidades con comportamiento real, no anémicas (H-17).

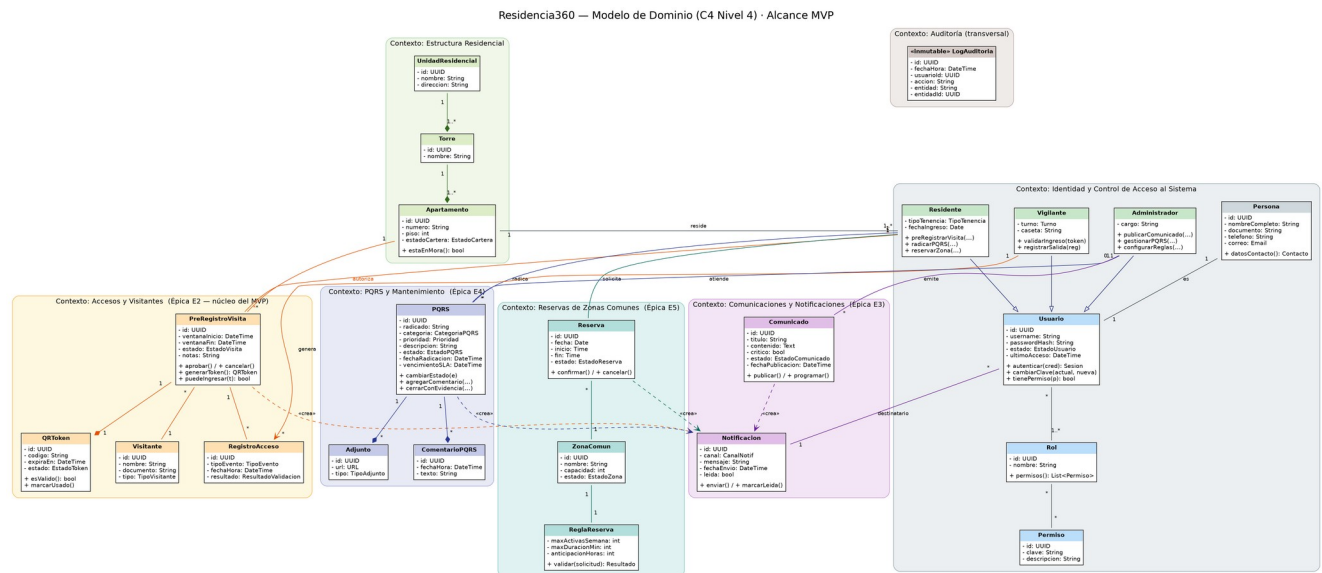


Figura 4. Modelo de dominio refinado por contextos delimitados (C4 nivel 4).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

Conexión con el diagrama de clases del E1. Este modelo es la evolución directa del diagrama de la primera entrega: conserva todas las entidades pertinentes del dominio (Usuario, Residente, Vigilante, Apartamento, PreRegistroVisita, QRToken, PQRS, Reserva, ZonaComun, Comunicado, Notificación, LogAuditoría) y corrige su formalización. Las enumeraciones tipan los estados que antes vivían sueltos en las reglas de negocio, conectando RN-02 y RN-09 con el modelo (H-05) y preparando las máquinas de estado del Entregable N.º 3.

Patrones visibles en el dominio. El fan-out «crea» desde Comunicado, PreRegistroVisita, PQRS y Reserva hacia Notificación anticipa el patrón Observer; la generalización de roles sobre Usuario habilita el polimorfismo del patrón Strategy de autorización; y la separación de ReglaReserva como objeto de política habilita su validación intercambiable.

4. Patrones de diseño (GoF)

Esta sección justifica la selección y aplicación de cinco patrones de diseño del catálogo GoF (Gamma, Helm, Johnson y Vlissides) dentro del alcance del MVP. Cada patrón responde a una necesidad real del sistema —no a un ejercicio académico— y se relaciona con una funcionalidad priorizada y con un componente concreto del nivel 3 del modelo C4. La selección evita deliberadamente la sobreingeniería: se eligen patrones cuya implementación es viable en tiempo y complejidad, y que no se solapan entre sí.

Los cinco patrones seleccionados son: Strategy (políticas de SLA y de reglas de reserva intercambiables), Observer (notificaciones desacopladas ante eventos de dominio), State (ciclos de vida de visita y PQRS), Repository con inversión de dependencias (aislamiento de la persistencia) y Factory Method (creación de notificaciones según canal). La Tabla 3 los sintetiza; los apartados 4.1 a 4.5 los desarrollan en profundidad.

Tabla 3. Resumen de los cinco patrones GoF seleccionados.

Patrón	Tipo	Necesidad del MVP	Componente C4 (N3)
Strategy	Comportamiento	Calcular el SLA según prioridad y validar reglas de reserva configurables sin condicionales rígidos.	PqrsService · ReservaService (Validadores/Reglas)
Observer	Comportamiento	Notificar a varios destinatarios cuando ocurre un evento, sin acoplar el emisor al receptor.	NotificacionService
State	Comportamiento	Gobernar las transiciones legales del ciclo de vida de la visita y de la PQRS.	Modelo de dominio (PreRegistroVisita, PQRS)
Repository	Estructural (con DIP)	Aislar el dominio de la tecnología de persistencia mediante interfaces.	Repositorios (IVisitaRepo, IPqrsRepo...)
Factory Method	Creación	Crear la notificación adecuada según el canal (In-App, Email) de forma extensible.	NotificacionService (NotificacionFactory)

4.1 Strategy — políticas de SLA y de reglas de reserva

Problema que resuelve. El sistema debe calcular la fecha de vencimiento de una PQRS según su prioridad (RN-09: Alta 24 h, Media 72 h, Baja 7 días) y validar las reservas según reglas configurables (RN-06 anticipación, RN-07 máximo semanal, RN-08 mora). Implementar esto con condicionales encadenados (if prioridad == ALTA ... else if ...) genera un método frágil que hay que modificar cada vez que cambia una política, violando el principio abierto/cerrado.

Solución con el patrón. Se define una familia de estrategias intercambiables tras una interfaz común. Para el SLA, una interfaz PolíticaSLA con una implementación por prioridad; para las reservas, una interfaz ReglaReserva cuyas validaciones concretas se componen. El servicio selecciona la estrategia en tiempo de ejecución y delega en ella.

```
interface PoliticaSLA { calcularVencimiento(creacion: DateTime): DateTime }
class SlaAlta implements PoliticaSLA { /* +24h */ }
class SlaMedia implements PoliticaSLA { /* +72h */ }
class SlaBaja implements PoliticaSLA { /* +7d */ }

class PqrsService {
    private politicas: Map<Prioridad, PoliticaSLA>;
    radicar(dto) {
        const politica = this.politicas.get(dto.prioridad); // selección de estrategia
        pqrs.vencimientoSLA = politica.calcularVencimiento(now());
    }
}
```

Criterio	Análisis
Por qué fue elegido	Las políticas de SLA y de reserva son, por requisito, configurables y cambiantes; encapsularlas como estrategias permite modificarlas o añadir nuevas sin tocar el servicio.
Alternativas descartadas	(a) Condicionales encadenados: viola OCP y concentra la lógica. (b) Tabla de parámetros sin polimorfismo: sirve para valores, pero no para lógica de cálculo distinta. (c) Patrón Template Method: acopla las variantes por herencia en lugar de composición.
Relación con el MVP	Directa: PQRS (RF-13/14) y reservas (RF-15/16) son funcionalidades del MVP, y sus reglas (RN-06 a RN-09) son explícitas.
Beneficios concretos	Cumple OCP; aísla cada política para pruebas unitarias independientes; permite cambiar SLAs por configuración sin recompilar la lógica.
Riesgos y mitigación	Proliferación de clases pequeñas. Se mitiga limitando las estrategias a las tres prioridades reales y registrando las implementaciones en un único punto (inyección).
Coherencia arquitectónica	Reside en la capa de dominio/validadores del nivel 3 del C4; el servicio de aplicación solo orquesta.

4.2 Observer — notificaciones ante eventos de dominio

Problema que resuelve. Múltiples acciones del sistema deben disparar notificaciones (RF-19): aprobar una visita, registrar un ingreso, radicar o responder una PQRS, confirmar una reserva, publicar un comunicado crítico (RN-11). Si cada servicio invocara directamente el envío de correo y la persistencia de la notificación, quedaría acoplado a esos detalles y se repetiría la lógica.

Solución con el patrón. Los servicios de dominio publican eventos; el NotificacionService actúa como observador suscrito a ellos. Cuando ocurre un evento, el observador decide el canal, construye el mensaje y lo envía, sin que el emisor conozca al receptor. En el modelo de dominio (C4 nivel 4) esto se visualiza como el fan-out «crea» desde Comunicado, PreRegistroVisita, PQRS y Reserva hacia Notificación.

```
interface EventoObserver { onEvento(e: EventoDominio): void }

class NotificacionService implements EventoObserver {
    onEvento(e) {
        const notif = this.factory.crear(e.canalPreferido, e.destinatario, e.mensaje);
        notif.enviar(); this.repo.guardar(notif);
    }
}

class PqrsService {
    private observers: EventoObserver[];
    radicar(dto){ /* ... */ this.publicar(new PqrsRadicada(pqrs)); } // notifica sin
acoplar
}
```

Criterio	Análisis
Por qué fue elegido	Desacopla la generación de eventos de su tratamiento; permite añadir nuevos tipos de notificación o canales sin modificar los servicios emisores.
Alternativas descartadas	(a) Llamada directa al envío desde cada servicio: alto acoplamiento y duplicación. (b) Cola de mensajes/broker: potente pero sobredimensionado para el MVP (sobreingeniería). (c) Patrón Mediator: añade un coordinador central innecesario para esta escala.
Relación con el MVP	Directa: las notificaciones (RF-19, RU-11) cruzan visitas, PQRS, reservas y comunicados, todos en el MVP.
Beneficios concretos	Cumple SRP (cada servicio hace lo suyo) y OCP (nuevos observadores sin tocar emisores); centraliza la política de notificación y su auditoría.
Riesgos y mitigación	Flujos difíciles de seguir si hay muchos observadores. Se mitiga manteniendo un único observador (NotificacionService) en el MVP y documentando los eventos.
Coherencia arquitectónica	El NotificacionService es un componente explícito del nivel 3 del C4, suscrito a los servicios de aplicación.

4.3 State — ciclos de vida de la visita y de la PQRS

Problema que resuelve. La visita (RN-02) y la PQRS (RN-09/10) tienen ciclos de vida con transiciones legales e ilegales. Por ejemplo, una visita solo puede pasar a «Ingresada» desde «Aprobada» y dentro de la ventana horaria (RN-03); una PQRS solo se cierra desde «Resuelta» con evidencia (RN-10). Codificar estas reglas con banderas booleanas y condicionales dispersos es propenso a estados inválidos.

Solución con el patrón. Cada entidad delega su comportamiento dependiente del estado en un objeto-estado. Las transiciones válidas se definen en cada estado concreto; intentar una transición ilegal lanza una excepción de dominio. Las máquinas de estado de la sección 6 del Entregable N.º 3 son la especificación visual de este patrón.

```
interface EstadoVisita { validarIngreso(v, token): void; cancelar(v): void }
class Aprobada implements EstadoVisita {
    validarIngreso(v, token){
        if(!token.esValido() || !v.enVentana()) throw new IngresoDenegado();
        v.transicionarA(new Ingresada());          // transición controlada
    }
}
class Ingresada implements EstadoVisita {
    validarIngreso(){ throw new TransicionIlegal('ya ingresó'); }
}
```

Criterio	Análisis
Por qué fue elegido	Hace imposible (por diseño) una transición ilegal y concentra en un solo lugar las reglas de cada estado, en vez de dispersarlas.
Alternativas descartadas	(a) Enum + switch en cada método: reaparece en muchos sitios y viola OCP. (b) Banderas booleanas: permiten combinaciones inválidas. (c) Motor de workflow externo: sobreingeniería para dos ciclos de vida.
Relación con el MVP	Directa: visitas (RF-04/06/07) y PQRS (RF-13/14) son el núcleo del MVP y sus estados están normados por reglas.
Beneficios concretos	Elimina estados inválidos; cada estado es probable de forma aislada; las transiciones quedan documentadas y trazables a RN-02, RN-03, RN-10.
Riesgos y mitigación	Más clases por entidad. Se mitiga aplicándolo solo a las dos entidades con ciclo de vida real (visita y PQRS), no a todas.
Coherencia arquitectónica	Vive en el modelo de dominio (C4 nivel 4) y se especifica con las máquinas de estado del E3 (coherencia entre artefactos).

4.4 Repository (con inversión de dependencias)

Problema que resuelve. Los servicios de aplicación necesitan leer y escribir entidades, pero no deben depender del motor de base de datos ni del ORM concreto. Si el servicio construyera consultas SQL directamente, quedaría acoplado a la tecnología y sería imposible de probar sin una base de datos real.

Solución con el patrón. Se define una interfaz de repositorio por agregado (IVisitaRepo, IPQRSRepo, IReservaRepo, IUsuarioRepo, IAuditRepo) en la capa de dominio; las implementaciones concretas con el ORM viven en la capa de persistencia. Los servicios dependen de la abstracción, no de la implementación (Dependency Inversion Principle).

```
// dominio (no conoce la BD)
interface IVisitaRepo { buscarPorToken(c: string): PreRegistroVisita; guardar(v): void }

// persistencia (detalle reemplazable)
class VisitaRepoPostgres implements IVisitaRepo { /* usa ORM */ }

class AccesoService {
  constructor(private repo: IVisitaRepo) {}      // inyección por interfaz (DIP)
}
```

Criterio	Análisis
Por qué fue elegido	Aísla el dominio de la infraestructura, habilita pruebas con repositorios en memoria y permite cambiar el motor sin tocar la lógica.
Alternativas descartadas	(a) Active Record (la entidad se persiste a sí misma): mezcla dominio y persistencia, dificulta pruebas. (b) DAO sin interfaz: acopla a la implementación. (c) Acceso directo al ORM desde servicios: viola DIP.
Relación con el MVP	Transversal: todos los casos de uso del MVP leen o escriben datos a través de repositorios.
Beneficios concretos	Cumple DIP y DIP habilita pruebas unitarias del dominio sin BD; desacopla del proveedor; centraliza el acceso a datos por agregado.
Riesgos y mitigación	Capa de indirección adicional. Se justifica por la testabilidad y el aislamiento; se evita crear repositorios para entidades sin identidad propia.
Coherencia arquitectónica	Es el componente «Repositorios» del nivel 3 del C4 y la frontera que conecta el contenedor API con la base de datos en el nivel 2.

4.5 Factory Method — creación de notificaciones por canal

Problema que resuelve. Una notificación puede entregarse por distintos canales (In-App y Email en el MVP; SMS u otros a futuro). Crear la instancia adecuada con condicionales dentro del `NotificacionService` acoplaría la creación al uso y obligaría a modificar el servicio cada vez que se añade un canal.

Solución con el patrón. Se delega la creación a una fábrica que, dado el canal, devuelve la implementación concreta de `Notificacion`. El observador (4.2) usa la fábrica sin conocer las clases concretas, lo que permite añadir canales nuevos creando una clase y registrándola, sin tocar el servicio.

```
interface Notificacion { enviar(): void }
class NotificacionInApp implements Notificacion { enviar(){/* persiste in-app */} }
class NotificacionEmail implements Notificacion { enviar(){/* SMTP */} }

class NotificacionFactory {
    crear(canal: CanalNotif, destino, msg): Notificacion {
        switch(canal){ case 'EMAIL': return new NotificacionEmail(destino,msg);
                       default:      return new NotificacionInApp(destino,msg); }
    }
}
```

Criterio	Análisis
Por qué fue elegido	Separa la decisión de qué objeto crear de la lógica que lo usa; añadir un canal no modifica al consumidor (OCP).
Alternativas descartadas	(a) Condicionales de creación dispersos: duplican la decisión. (b) Abstract Factory: pensado para familias de objetos relacionados; aquí solo varía un producto, sería excesivo. (c) Builder: aporta valor cuando la construcción es compleja por pasos, no es el caso.
Relación con el MVP	Directa: las notificaciones del MVP usan In-App y correo; la fábrica deja preparado el canal SMS como extensión.
Beneficios concretos	Cumple OCP y SRP; centraliza la creación; simplifica el observador, que pasa a depender solo de la interfaz <code>Notificacion</code> .
Riesgos y mitigación	Un switch en la fábrica sigue creciendo con cada canal; es aceptable por estar aislado en un único punto y es sustituible por un registro si crece.
Coherencia arquitectónica	Forma parte del componente <code>NotificacionService</code> (nivel 3) y colabora con el patrón <code>Observer</code> (4.2), sin solaparse: <code>Observer</code> decide cuándo notificar, <code>Factory Method</code> decide qué objeto notificación crear.

4.6 No solapamiento y coherencia del conjunto

Los cinco patrones cubren responsabilidades distintas y complementarias, sin redundancia: `Strategy` encapsula algoritmos variables (cómo se calcula algo); `State` encapsula comportamiento dependiente del estado (qué se puede hacer ahora); `Observer` desacopla la reacción a eventos (cuándo avisar); `Factory Method` encapsula la creación (qué objeto instanciar); y `Repository` aísla la persistencia (dónde se guardan los datos). `Observer` y `Factory`

Method colaboran en el flujo de notificación sin solaparse, y Strategy y State, aunque ambos son de comportamiento, operan sobre ejes diferentes (algoritmo vs. estado). Esta separación confirma que la selección evita la sobreingeniería y mantiene la coherencia con la evolución del proyecto.

5. Prototipo funcional

El prototipo funcional implementa, en su flujo principal y sin caminos alternativos, las historias de usuario priorizadas del MVP. Toma como base el prototipo Figma del Entregable N.º 1 y las historias y el story mapping ya consolidados, de modo que la funcionalidad demostrada se corresponde uno a uno con lo especificado.

5.1 Historias del MVP implementadas (flujo principal)

Historia	Funcionalidad del flujo principal	Pantallas
HU-01	Inicio de sesión con usuario y contraseña; redirección al panel según rol.	S-01 Login · S-02 Panel
HU-03	Pre-registro de visitante: datos, ventana horaria y generación de QRToken.	S-03 Pre-registro
HU-05	Validación de ingreso en portería leyendo el QR y registrando la entrada.	S-04 Portería
HU-07	Publicación de comunicado segmentado con notificación a residentes.	S-05 Comunicados
HU-09	Radicación de PQRS con categoría, prioridad, descripción y adjuntos.	S-06 Nueva PQRS
HU-11	Reserva de zona común consultando disponibilidad y aplicando reglas.	S-08 Reservas
HU-15	Consulta de reportes básicos filtrables por fecha.	S-12 Reportes

5.2 Tecnologías y ejecución

Tecnologías. Frontend en React (SPA responsive); backend en Node.js con Express exponiendo una API REST; base de datos PostgreSQL; autenticación basada en token (JWT). Estas tecnologías corresponden a los contenedores del nivel 2 del modelo C4.

Usuarios de prueba (para la sustentación). Se definirá un usuario por rol (residente, vigilante, administrador y consejo) con credenciales de demostración, cerrando el vacío señalado en la auditoría (H-28). El prototipo Figma de referencia permanece accesible para contraste visual.

Capturas y evidencias. Las capturas del prototipo funcional se incorporan en los anexos (§8). El prototipo Figma de origen se enlaza igualmente como soporte de la continuidad con el Entregable N.º 1.

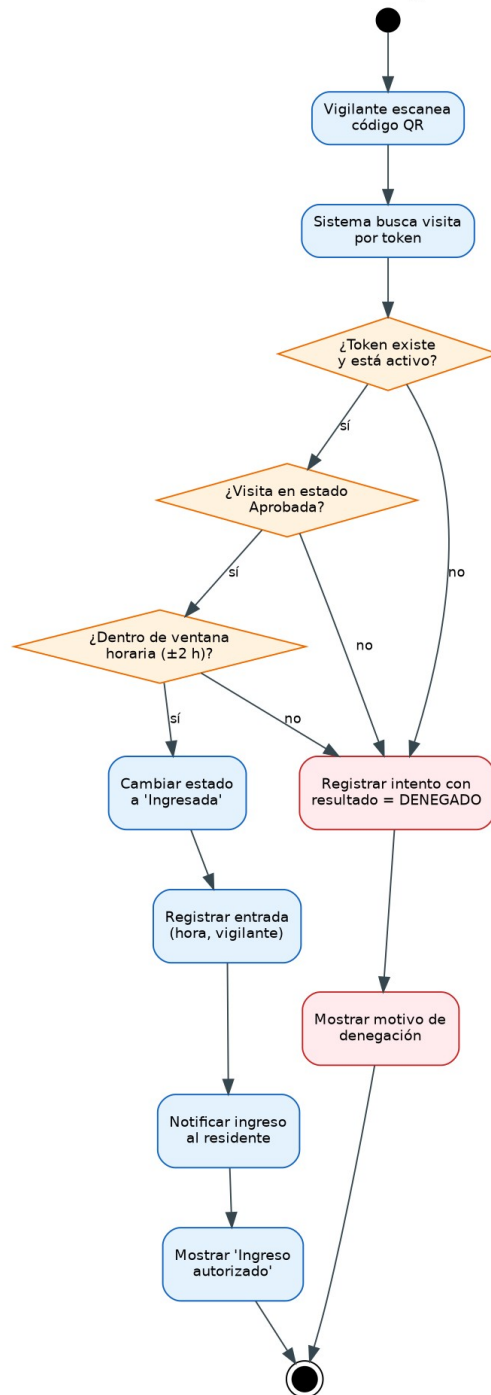
6. Diagramas de actividad del MVP

Se presentan los diagramas de actividad correspondientes a las historias de usuario implementadas. Cada diagrama representa el flujo principal con sus acciones, decisiones y procesos relevantes, y sus compuertas de decisión se trazan a las reglas de negocio definidas en el Entregable N.º 1.

6.1 HU-05 — Validar ingreso en portería

El flujo encadena tres validaciones: existencia y vigencia del QRToken, estado «Aprobada» de la visita y permanencia dentro de la ventana horaria. Solo si las tres se cumplen se registra la entrada y se notifica al residente; en caso contrario, se registra el intento con resultado denegado, conforme a la regla RN-03.

Diagrama de Actividades — HU-05: Validar ingreso en portería

**Figura 5.** Diagrama de actividad — HU-05 (validar ingreso).Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

6.2 HU-11 — Reservar zona común

El flujo valida en secuencia que el apartamento esté al día (RN-08), que la franja esté disponible, que se cumpla la anticipación mínima (RN-06) y que no se supere el máximo semanal (RN-07). Al superar todas las compuertas, crea la reserva y notifica su estado.

Diagrama de Actividades — HU-11: Reservar zona común

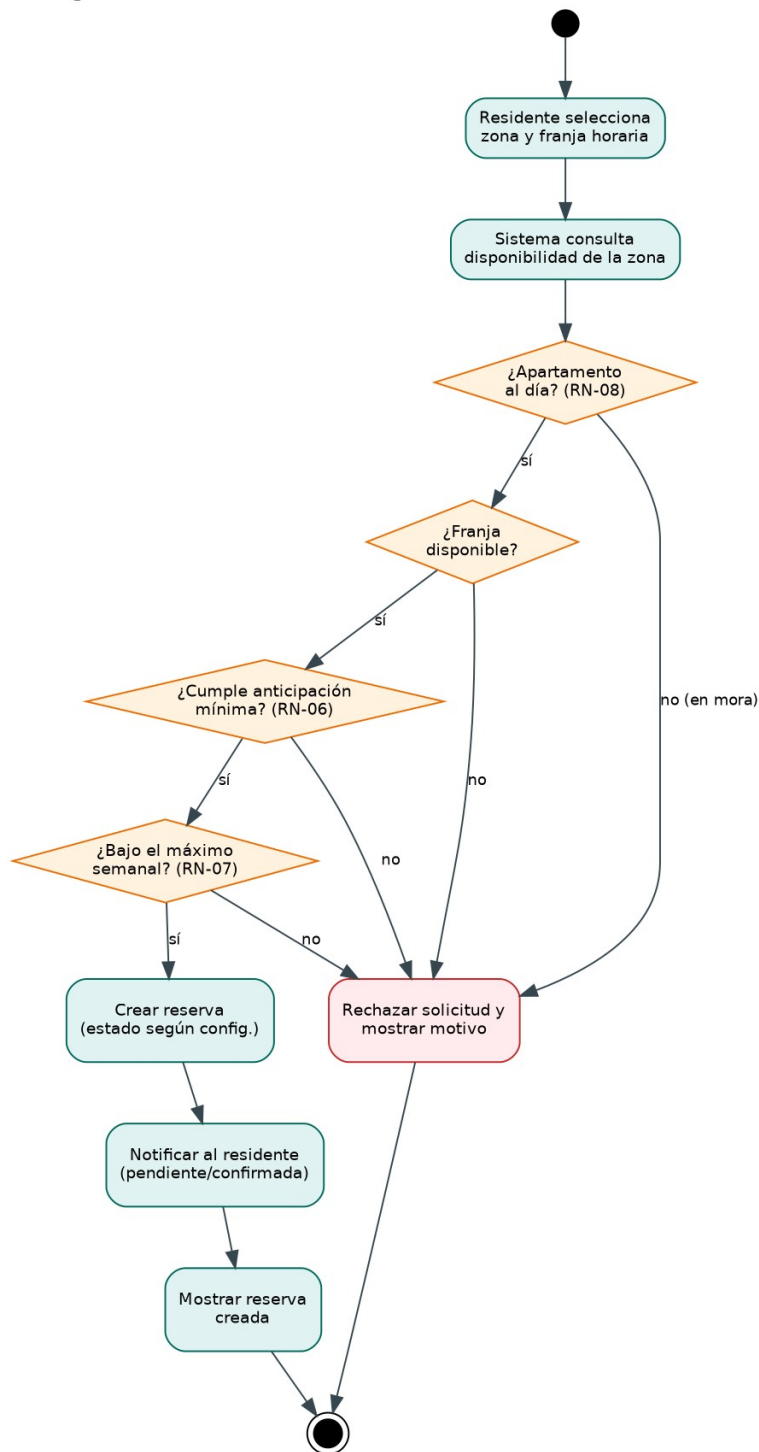


Figura 6. Diagrama de actividad — HU-11 (reservar zona común).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

6.3 HU-09 — Radicar PQRS

El flujo valida los campos obligatorios (RN-09); si están completos, genera el radicado, calcula el SLA según la prioridad —aplicando el patrón Strategy de §4.1—, persiste la PQRS en estado «Radicada» y notifica a administración mediante el patrón Observer de §4.2.

Diagrama de Actividades — HU-09: Radicar PQRS con adjuntos

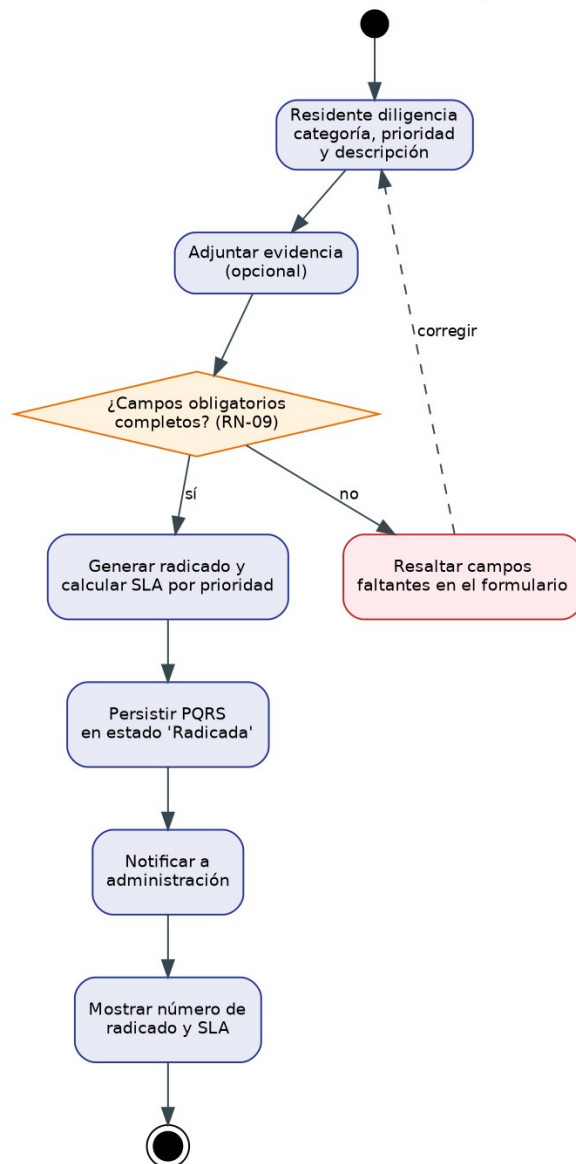


Figura 7. Diagrama de actividad — HU-09 (radicar PQRS).

Enlace editable: [\[pegar aquí el enlace al diagrama editable\]](#)

7. Aplicación de los principios SOLID

El análisis SOLID se realiza sobre componentes reales de la evolución del proyecto —los definidos en el nivel 3 del modelo C4 y en el modelo de dominio— y no sobre ejemplos aislados. Para cada principio se indica dónde se aplica en Residencia360, cómo y qué patrón de la sección 4 lo refuerza.

S — Responsabilidad única (SRP)

Dónde se aplica. Separación entre controladores, servicios de aplicación, entidades de dominio y repositorios (C4 nivel 3).

Cómo. Cada controlador solo traduce HTTP; cada servicio orquesta un caso de uso; cada entidad encapsula su comportamiento; cada repositorio solo persiste. El `NotificacionService` tiene la única responsabilidad de notificar, en vez de que cada servicio lo haga.

Patrón que lo refuerza. Observer (4.2) extrae la responsabilidad de notificar fuera de los servicios emisores.

O — Abierto/Cerrado (OCP)

Dónde se aplica. Cálculo de SLA, validación de reservas y creación de notificaciones.

Cómo. Añadir una nueva prioridad de SLA, una nueva regla de reserva o un nuevo canal de notificación se hace creando una clase nueva, sin modificar el código existente que la consume.

Patrón que lo refuerza. Strategy (4.1) y Factory Method (4.5) son las realizaciones directas de OCP en el sistema.

L — Sustitución de Liskov (LSP)

Dónde se aplica. Jerarquía de roles (Residente, Vigilante, Administrador como subtipos de Usuario) y las familias de estrategias y estados.

Cómo. Cualquier subtipo de Usuario puede usarse donde se espera un Usuario (p. ej. autenticarse); cualquier `PoliticaSLA` o `EstadoVisita` concreto es intercambiable por su interfaz sin romper el comportamiento esperado.

Patrón que lo refuerza. State (4.3) y Strategy (4.1): sus implementaciones respetan el contrato de la interfaz que sustituyen.

I — Segregación de interfaces (ISP)

Dónde se aplica. Interfaces de repositorio por agregado y la interfaz de observador.

Cómo. En lugar de un único repositorio gigante, se definen interfaces específicas (`IVisitaRepo`, `IPQRSRepo`, `IReservaRepo`...), de modo que cada servicio depende solo de las operaciones que necesita y no de un contrato inflado.

Patrón que lo refuerza. Repository (4.4): las interfaces pequeñas por agregado son la aplicación de ISP.

D — Inversión de dependencias (DIP)

Dónde se aplica. Frontera entre los servicios de aplicación y la persistencia.

Cómo. Los servicios dependen de abstracciones (interfaces de repositorio) y no de las implementaciones con ORM; estas se inyectan. El dominio no conoce la base de datos, lo que invierte la dirección de la dependencia hacia las abstracciones.

Patrón que lo refuerza. Repository con inyección (4.4) es la materialización de DIP; habilita además las pruebas del dominio sin base de datos.

Síntesis. Los cinco principios no se cumplen de forma abstracta sino a través de decisiones concretas y verificables en la arquitectura: la organización por capas del C4 sostiene SRP e ISP, y los patrones Strategy, Factory Method, State y Repository sostienen OCP, LSP y DIP. Existe, por tanto, una correspondencia explícita entre arquitectura, patrones y principios.

8. Anexos y referencias

8.1 Anexos (soportes del trabajo)

Los siguientes soportes acompañan el entregable. Los enlaces deben completarse con las URL reales del equipo antes de la entrega final.

Soporte	Enlace / Ubicación
Repositorio de código	[pegar URL del repositorio Git]
Prototipo Figma (origen E1)	https://rare-jeep-73888365.figma.site/login
Diagramas C4 (editables)	[pegar enlaces editables de los 4 niveles]
Máquinas de estado y secuencia	[pegar enlaces editables]
Diagrama integrado componentes/despliegue	[pegar enlace editable]
Plantilla de patrones GoF	Sección 4 (Tabla 3 y plantillas 4.1–4.5)
Plantilla de principios SOLID	Sección 7
Capturas del prototipo funcional	[adjuntar capturas]

8.2 Referencias

- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Martin, R. C. (2002). Agile Software Development: Principles, Patterns, and Practices. Prentice Hall. (Principios SOLID.)
- Brown, S. (2023). The C4 model for visualising software architecture. c4model.com.
- Patton, J. (2014). User Story Mapping: Discover the Whole Story, Build the Right Product. O'Reilly Media.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. (Repository, Active Record.)
- Object Management Group (2017). Unified Modeling Language (UML), versión 2.5.1.